Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №2

РОЗПАРАЛЕЛЕННЯ МНОЖЕННЯ МАТРИЦЬ

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

1. Мета роботи

Мета цієї роботи – дослідити методи множення матриць, реалізувати їх **послідовно** та **паралельно**, заміряти час виконання та проаналізувати ефективність паралельних обчислень.

---

2. Теоретична частина

2.1 Послідовне множення

Добуток двох матриць C=A×B

$$C[i][j] = \sum_{k=1}^{m} A[i][k] \cdot B[k][j]$$

Обчислювальна складність для квадратних матриць розмірності n×n:

$$O(n^3)$$

2.2 Паралельне множення

Ідея – розподілити обчислення рядків матриці результату між кількома потоками.

- При k потоках кожен отримує приблизно n/k рядків.
- Якщо n не кратне k, перші потоки обробляють на один рядок більше.
- Результати синхронізуються в головному потоці.

2.3 Закон Амдала

Навіть якщо розпаралелити обчислення, існують частини коду, які **залишаються послідовними** (ініціалізація, створення потоків, синхронізація).
Нехай:

- P — частка коду, що виконується паралельно;
- (1−P) — послідовна частина;
- k — кількість потоків.

Тоді максимальне прискорення за законом Амдала:

$$S_{max}(k) = \frac{1}{(1-P) + \frac{P}{k}}$$

Висновок: навіть при нескінченній кількості потоків максимальне прискорення обмежене величиною:

$$S_{max} = \frac{1}{1 - P}$$

Тобто, якщо лише 90% коду можна паралелити, то максимальне прискорення = 10×, навіть при тисячах потоків.

2.4 Метрики оцінки

- **Прискорення:**

$$S = \frac{T_{seq}}{T_{par}}$$

- **Ефективність:**

$$E = \frac{S}{k} \cdot 100\%$$

3. Алгоритм виконання

1. Генеруємо матриці розмірності n×n.
2. Виконуємо послідовне множення, заміряємо час.
3. Виконуємо паралельне множення:
    - ділимо рядки між потоками;
    - запускаємо потоки;
    - синхронізуємо результат.
4. Обчислюємо S, E, аналізуємо дані з точки зору закону Амдала.

5. Результати

Для матриць 1000x1000 та до 12 потоків

Я взяв дві великі матриці розміром 1000×1000 та виконав їх множення. Спочатку обчислення проводилися в одному потоці, а потім поступово збільшувалась кількість потоків до 12, щоб перевірити, як це впливає на швидкість виконання.

```
Введіть кількість рядків матриці A: 1000
Введіть кількість стовпців матриці A / рядків матриці B: 1000
Введіть кількість стовпців матриці B: 1000
Введіть максимальну кількість потоків для тестування: 12

=== Розмір матриць ===
A: 1000 × 1000
B: 1000 × 1000
Кількість процесорів: 12
Тестування до 12 потоків

Обчислення послідовного множення...
Час послідовного множення: 4112,00 ms
```

При роботі в одному потоці множення тривало понад 4 секунди. Це досить довго.

```
=== Результати тестування ===
Потоки  Час (ms)        Прискорення     Ефективність
==================================================
1       4112,00         1.00×           100.0%
2       2075,60         1,98×           99,1 %
3       1379,19         2,98×           99,4 %
4       1156,19         3,56×           88,9 %
5       1086,32         3,79×           75,7 %
6       992,33          4,14×           69,1 %
7       965,91          4,26×           60,8 %
8       884,60          4,65×           58,1 %
9       886,51          4,64×           51,5 %
10      828,76          4,96×           49,6 %
11      796,21          5,16×           46,9 %
12      776,14          5,30×           44,2 %
```

Коли використав 2 потоки, час скоротився майже вдвічі — до 2 секунд.
При 3 потоках результат став ще кращим — приблизно 1,4 секунди. Але з подальшим збільшенням кількості потоків прискорення вже не було таким значним.
Наприклад, при 12 потоках виконання зайняло 0,78 секунди, що відповідає лише 5,3-кратному прискоренню порівняно з одним потоком, замість очікуваного 12-кратного.

Чому так сталось:

Зі збільшенням кількості потоків виникають накладні витрати. Потоки починають конкурувати за ресурси — оперативну пам’ять, кеш процесора тощо. Крім того, з’являються витрати на синхронізацію та узгодження їх роботи. Існує також частина алгоритму, яку неможливо паралелізувати — вона виконується послідовно, що обмежує максимальне прискорення (закон Амдала).

Висновок:

Найефективнішими виявилися 2–3 потоки — вони майже ідеально розподілили роботу. При 6–8 потоках швидкість зростала, але вже не так суттєво. При 12 потоках час виконання був найменший, проте ефективність кожного потоку становила лише близько 44%.
Отже, для таких задач оптимально використовувати 6–8 потоків: час обчислення майже мінімальний, але ресурси при цьому витрачаються значно раціональніше.

```
=== Висновки ===
Найшвидша кількість потоків: 12
Найкращий час: 776,14 ms
Прискорення: 5,30×
Ефективність: 44,2 %

=== Аналіз паралелізації ===
Оцінена паралельна частка: 88,5 %
Теоретичний максимум прискорення: 8,70×
Загальна кількість операцій множення: 1 000 000 000
```